

Trabalho Prático

Sistemas Operativos II

2024/2025



Realizado por:

Martim Oliveira Jardim Freitas – 2021145193

Rodrigo Gonçalves Glória – 2022150127

Introdução

O presente relatório descreve a implementação do trabalho prático da unidade curricular de Sistemas Operativos 2, que consiste no desenvolvimento de um jogo de identificação de palavras, envolvendo múltiplos processos e mecanismos de comunicação e sincronização em sistemas operativos. O jogo, concebido para ser executado em ambiente Windows, requer a interação entre quatro programas principais: o **árbitro**, responsável pela gestão central do jogo; o **jogoui**, que serve como interface para os jogadores humanos; o **bot**, um jogador automatizado;

O objetivo principal deste trabalho é aplicar conceitos fundamentais de sistemas operativos, como comunicação entre processos (utilizando *named pipes*), memória partilhada, sincronização e gestão de recursos. Este trabalho não só demonstra a aplicação prática dos conceitos teóricos lecionados, mas também evidencia a importância do trabalho em equipa e da tomada de decisões técnicas fundamentadas. A implementação bem-sucedida do jogo requer não apenas o cumprimento dos requisitos funcionais, mas também a garantia de robustez, eficiência e usabilidade.

Estrutura de Dados

- Estrutura Jogador

hPipe: handle do named pipe para comunicação com o jogador.

username: nome do jogador.

is_bot: indica se é um bot.

pontuacao: pontuação atual.

tempo_reacao: tempo de reação (usado para bots).

continua/ativo: flags de estado do jogador.

estado: ponteiro para o estado global do jogo.

- Estrutura EstadoJogo

jogadores[MAX_JOGADORES]: vetor com o estado de todos os jogadores ligados (humanos e bots).

num_jogadores: número atual de jogadores conectados.

hMutexJogadores: mutex para proteger o acesso à lista de jogadores e contagem.

jogoAtivo: flag que indica se o jogo está ativo (TRUE) ou encerrado (FALSE).

ritmo: intervalo (em segundos) entre sorteios de letras.

max_letras: número máximo de letras visíveis em simultâneo.

num_palavras: número total de palavras no dicionário.

dicionario[MAX Palavra][MAX TAMANHO PALAVRA]: lista de palavras válidas.

pMem: ponteiro para a memória partilhada com letras visíveis.

hMap: handle do mapeamento de ficheiro da memória partilhada.

hMutexSM: mutex para proteger o acesso à memória partilhada.

hEventoLetras: evento que sinaliza alterações nas letras visíveis.

tdLetras: estrutura de controlo da thread de sorteio de letras.

hEventoJogadores: evento que sinaliza alterações na lista de jogadores.

- BotState

hPipe: handle do named pipe entre o bot e o árbitro.

username[MAX_USERNAME]: nome do bot no jogo.

pShm: ponteiro para a memória partilhada com as letras visíveis.

hMutex: mutex para acesso exclusivo à memória partilhada.

hEvento: evento que sinaliza atualizações nas letras.

hMap: handle do mapeamento da memória partilhada.

pontuacao: pontuação atual do bot.

tempo_reacao: tempo de espera antes de tentar formar uma palavra.

continua: flag que indica se o bot continua em execução.

- SData

letras[MAXLETRAS_LIMITE]: vetor de letras atualmente visíveis no jogo.

tempo[MAXLETRAS_LIMITE]: tempo associado a cada letra visível.

maxLetras: número máximo de letras visíveis em simultâneo.

ritmo: intervalo (em segundos) entre sorteios de novas letras.

- Dados

username[TAM_BUFFER]: nome do jogador/bot que envia ou recebe a mensagem.

comando[TAM_BUFFER]: comando enviado (ex: :msg, :sair, :pont).

msg[TAM_BUFFER]: conteúdo da mensagem (ex: palavra jogada ou resposta).

- dThreadELetras e dThreadLeLetras

hMutex: mutex que protege o acesso à memória partilhada das letras.

hEvento: evento que sinaliza alterações nas letras.

pShm: ponteiro para a memória partilhada com letras e tempos.

continua: flag que controla a execução da thread.

Árbitro (arbitro.c)

O programa arbitro é o processo central do jogo, responsável por coordenar a comunicação entre os jogadores (humanos ou bots), manter o estado global e controlar o ciclo de jogo.

1. Estruturas principais

- EstadoJogo: guarda todo o estado partilhado do jogo, incluindo:
 - Lista de jogadores (jogadores[MAX_JOGADORES]);
 - Estado do jogo (ativo/inativo);
 - Ritmo e número máximo de letras;
 - Dicionário carregado;
 - Ponteiro para a memória partilhada (SDATA* pMem);
 - Handles de sincronização (mutexes, eventos).
- Jogador: estrutura individual com dados do jogador como pipe, nome, pontuação, tipo (bot ou humano) e flags de estado.

2. Comunicação e sincronização

- A comunicação com os jogadores é feita via **named pipes**, com **overlapped I/O** para permitir leitura e escrita assíncrona.
- A memória partilhada (CreateFileMapping + MapViewOfFile) armazena as letras visíveis e respetivos tempos.
- Dois **mutexes** (hMutexJogadores, hMutexSM) protegem o acesso concorrente às estruturas dos jogadores e memória partilhada, respetivamente.
- Dois **eventos** (hEventoJogadores, hEventoLetras) sinalizam alterações no estado dos jogadores e das letras.

3. Inicialização

- As configurações iniciais (ritmo e número máximo de letras) são lidas do **registo do Windows**. Se não existirem, são usados valores por omissão.
- O dicionário é carregado a partir de um ficheiro de texto (dicionario.txt) e armazenado na estrutura EstadoJogo.

4. Threads

- ThreadLetras: responsável por sortear periodicamente novas letras e atualizá-las na memória partilhada.
- ThreadAdmin: permite ao administrador introduzir comandos pela consola para gerir o jogo (listar, excluir, iniciar bots, alterar ritmo, encerrar jogo).
- TratarJogador: criada dinamicamente para cada jogador. Recebe comandos via pipe e responde com base nas regras do jogo (verificação de palavras, pontuação, listagem, saída).

5. Funcionalidades adicionais (não mencionadas no enunciado)

- **Validação de nomes:** nomes não podem começar por “bot” e devem ser únicos. Esta verificação é feita na conexão.
- **Verificação de palavras:** inclui validação contra o dicionário e contra letras disponíveis (sem reutilização). Palavras incorretas penalizam o jogador.
- **Mensagens broadcast:** algumas mensagens (como acertos e erros) são enviadas a todos os jogadores, com exceção do jogador excluído quando aplicável.
- **Encerramento limpo:** todos os recursos (mutexes, eventos, memória mapeada, pipes) são libertados no final.

6. Robustez

- Várias proteções contra erros de I/O (ex: pipes, falhas de leitura/escrita).
- Controlo de concorrência rigoroso com uso apropriado de mutexes e eventos.
- Utilização de I/O overlapped para garantir responsividade mesmo com múltiplos jogadores.

Bot (bot.c)

O programa bot representa um jogador automático no jogo, capaz de formar palavras com base nas letras disponíveis na memória partilhada. O seu comportamento é autónomo e configurado aleatoriamente a cada execução.

1. Estrutura principal

- BotState: estrutura interna que guarda o estado do bot:
 - Comunicação com o árbitro (hPipe);
 - Memória partilhada (pShm, hMap);
 - Mecanismos de sincronização (hMutex, hEvento);
 - Parâmetros de jogo (pontuação, tempo de reação, etc.).

2. Comunicação com o árbitro

- Estabelecida através de um **named pipe**, em modo **overlapped** para suporte de I/O assíncrona.
- Ao iniciar, o bot envia uma mensagem :login com o seu nome ao árbitro.
- Cria uma thread auxiliar (ThreadLeitura) para ler mensagens recebidas do árbitro.

3. Integração com memória partilhada

- O bot acede à **memória partilhada** (letras e tempos) via OpenFileMapping e MapViewOfFile.
- Sincroniza o acesso às letras com OpenMutex e é notificado de alterações por OpenEvent.

4. Comportamento automático

- Um tempo de reação aleatório entre 5 e 30 segundos é atribuído ao bot.
- Após cada intervalo, o bot:
 1. **Lê as letras atuais** da memória partilhada.
 2. **Percorre o dicionário local** em busca da primeira palavra válida que consiga formar.
 3. **Envia essa palavra** ao árbitro com o comando :msg.

5. Carregamento do dicionário

- O dicionário é carregado diretamente do ficheiro dicionario.txt usando **memory-mapped files**, uma técnica eficiente que evita leitura linha a linha convencional.
- A conversão para Unicode é feita se necessário (com MultiByteToWideChar).

6. Processamento de mensagens recebidas

- O bot interpreta comandos vindos do árbitro:
 - :pont: atualiza e imprime a pontuação atual;
 - :sair: encerra o ciclo de jogo.

7. Robustez e encerramento

- Todos os handles (pipe, mutex, evento, mapeamento, threads) são corretamente libertados no final.
- O bot verifica constantemente se há mensagens pendentes via PeekNamedPipe, mesmo fora da thread de leitura.

Aspetos não explicitamente indicados no enunciado

- **Autonomia total:** o bot funciona sem intervenção externa após ser iniciado.
- **Gestão paralela de I/O:** uso de overlapped I/O permite que o bot seja responsivo sem bloquear.
- **Deteção de comandos :sair em vários pontos:** o bot pode ser encerrado remotamente em qualquer fase do ciclo.
- **Dicionário local independente:** cada bot carrega o seu próprio dicionário local

Jogador UI (jogoui.c)

O jogoui é o cliente de linha de comandos utilizado por jogadores humanos para interagir com o jogo. Permite enviar palavras e comandos ao árbitro através de um **named pipe** e receber respostas.

1. Comunicação com o árbitro

- O programa estabelece ligação ao árbitro através do named pipe PIPE_NAME, em modo overlapped.
- Envia uma estrutura DADOS com o comando :login e o nome do jogador logo após a conexão.
- Usa overlapped I/O para envio de comandos e receção de mensagens de forma não bloqueante.

2. Estrutura de dados

- Utiliza a estrutura DADOS definida no projeto para comunicar:
 - username: nome do jogador.
 - comando: tipo de ação (e.g., :msg, :sair, :pont, :jogs).
 - msg: conteúdo da mensagem (palavra jogada ou resposta textual).

3. Thread de leitura

- Uma thread separada (ThreadLeitura) é criada para receber mensagens enviadas pelo árbitro, como feedback de jogadas, pontuação ou mensagens de broadcast.
- As mensagens recebidas são imediatamente apresentadas no terminal.

4. Ciclo principal do jogador

- Após a ligação, o jogador pode escrever:
 - **Palavras válidas:** são enviadas como comando :msg.
 - **Comandos especiais:**
 - :pont: solicita a pontuação atual.
 - :jogs: solicita a lista de jogadores conectados.
 - :sair: termina a sessão.

5. Aspetos adicionais (não indicados no enunciado)

- **Interface simples e interativa:** apresenta sugestões de comandos e aguarda input do utilizador com prompt >.
- **Overlapped I/O consistente:** tanto para escrita como leitura, incluindo uso de Event e WaitForSingleObject.
- **Gestão de encerramento segura:** ao usar :sair, o jogador envia o comando ao árbitro, espera resposta e liberta corretamente os recursos.
- **Robustez de conexão:** o cliente aguarda o árbitro estar ativo antes de se ligar (loop com Sleep + CreateFile).

ID	Descrição funcionalidade / requisito	Estado no código
1. Funcionalidades básicas do jogo		
1.1	Jogo de identificação de palavras com letras aleatórias	Implementado
1.2	Múltiplos jogadores (até 20)	Implementado
1.3	Jogo inicia com 2 jogadores e para com 1	Implementado
1.4	Ações dos jogadores: juntar-se, indicar palavra, ver pontuação, sair	Implementado
2. Mecanismos do árbitro		
2.1	Gerenciamento de letras (sorteio, substituição)	Implementado
2.2	Validação de palavras contra dicionário	Implementado
2.3	Sistema de pontuação (1 ponto/letra para palavras válidas, - 0.5/letra para inválidas)	Implementado
2.4	Remoção de letras usadas em palavras válidas	Implementado
3. Comunicação e sincronização		
3.1	Comunicação árbitro-jogadores via named pipes	Implementado
3.2	Memória partilhada para letras visíveis	Implementado
3.3	Mecanismos de sincronização (mutex, eventos)	Implementado
4. Interface e administração		
4.1	Interface de administração do árbitro	Implementado
4.2	Interface do jogoi em consola	Implementado

4.3	Bot como jogador automático	Implementado
4.4	Painel gráfico (opcional)	Não implementado
5. Configurações e persistência		
5.1	Configurações (MAXLETRAS, RITMO) no registry	Implementado
5.2	Validação de MAXLETRAS ≤ 12	Implementado
6. Funcionalidades extras		
6.1	Broadcast de mensagens para jogadores	Implementado
6.2	Detecção de vencedor ao encerrar	Implementado
6.3	Suporte a Unicode	Implementado
7. Itens não implementados		
7.1	Programa painel gráfico	Não implementado